

DJEZZY NATIONAL SOC PLATFORM

Comprehensive Implementation Guide

Step-by-Step Deployment Procedures for 100% On-Premises Installation

Document Type	Technical Implementation Manual
Version	1.0 - Production Ready
Classification	INTERNAL USE - TECHNICAL OPERATIONS
Deployment Model	100% On-Premises (Air-Gap Capable)
Target Audience	Platform Engineers, DevOps, System Administrators
Effective Date	July 29, 2026
Review Cycle	Quarterly or Major Version Change

Document Scope

This Implementation Guide provides detailed step-by-step procedures for deploying the Djeczy National Security Operations Center (SOC) Platform in a 100% on-premises environment. It covers every phase from initial facility preparation through production go-live, including operating system installation, container runtime configuration, network setup, database deployment, security tool integration, application deployment, validation testing, and operational handoff procedures. Each section includes exact commands, configuration file templates, expected outputs, and troubleshooting guidance for common issues encountered during deployment.

How to Use This Guide

- Sequential Execution: Sections are ordered to match the logical deployment sequence. Complete each section fully before proceeding to the next.
- Prerequisites: Each section begins with prerequisites that must be satisfied before starting. Verify all prerequisites are met before beginning procedures.
- Command Blocks: Commands shown in dark green boxes should be executed exactly as written unless noted otherwise. Copy-paste carefully to avoid typos.
- Validation Steps: Each major procedure includes validation commands confirming successful completion. Always execute validation steps before proceeding.
- Troubleshooting: Common issues and resolutions appear at the end of each section. Consult these before escalating to senior engineers or vendor support.

Table of Contents

Section	Title	Page
1	Pre-Deployment Requirements & Checklists	3
2	Operating System Installation & Hardening	6
3	Container Runtime & Orchestration Setup	10
4	Network Configuration & Segmentation	14
5	Database Layer Deployment (PostgreSQL/Kafka/Redis)	18
6	Elasticsearch Cluster Deployment	24
7	SIEM Stack Deployment (Wazuh/Elasticsearch/Kibana)	28
8	Network Security Monitoring (Suricata/Zeek/Arkime)	33
9	Endpoint Detection & Response (GRR/Osquery)	38
10	SOAR Platform (TheHive/Cortex) Deployment	42
11	Threat Intelligence (MISP/OpenCTI) Setup	46
12	Vulnerability Management (OpenVAS/DefectDojo)	50
13	Monitoring & Observability Stack	54
14	API Gateway & Frontend Application	58
15	Integration Testing & Validation	62
16	Operational Runbooks & Procedures	66
17	Troubleshooting Guide	72
A	Appendix: Configuration Reference	78
B	Appendix: Command Quick Reference	82

1. Pre-Deployment Requirements & Checklists

Before beginning any installation activities, ensure all prerequisite conditions are satisfied. This section provides comprehensive checklists covering hardware readiness, software artifacts, network connectivity, personnel preparation, and documentation requirements. Complete each checklist in full and obtain sign-off from the designated authority before proceeding to Section 2 (Operating System Installation).

1.1 Hardware Prerequisites Checklist

Item	Specification	Status	Verified By	Date
Primary Site Servers (14)	All servers received and inspected	☐		
	Serial numbers recorded in CMDB	☐		
	Firmware updated to latest stable version	☐		
	Physical damage inspection passed	☐		
	All drive bays populated per spec	☐		
Network Infrastructure	Spine switches installed and powered	☐		
	Leaf switches installed and powered	☐		
	Fiber optic cables terminated and tested	☐		
	Copper cables terminated and tested	☐		
	Optics (SFP/SFP+/QSFP) available	☐		
Power Infrastructure	UPS systems installed and tested	☐		
	ATS/transfer switch functional	☐		
	PDU's installed and labeled	☐		
	Power draw test completed	☐		
Cooling & Environment	CRAC/CRAH units operational	☐		
	Temperature sensors calibrated	☐		
	Hot/cold aisle containment installed	☐		
	Fire suppression system armed	☐		
Rack Infrastructure	Server racks assembled and leveled	☐		
	Cable management installed	☐		
	Grounding verified (<1 ohm)	☐		

Table 1.1: Hardware Prerequisites Checklist

1.2 Software Artifacts Inventory

The following software packages must be obtained and staged on secure media (encrypted USB drives or internal network share) prior to beginning installation. For air-gapped deployments, download all packages beforehand using an internet-connected workstation, then transfer via secure media to the

isolated environment. Verify checksums (SHA256) for each package against vendor-provided values to ensure integrity.

Software Component	Version	Source	Format	Checksum Verify
Rocky Linux 9.4	9.4-20240508	rockylinux.org	ISO	[]
Docker CE	27.1.x	docker.com	RPM/RPM	[]
Docker Compose	v2.27.x	github.com	Binary	[]
PostgreSQL 16	16.4-1	postgresql.org	RPM	[]
Apache Kafka 3.6	3.6.1	kafka.apache.org	TGZ	[]
Apache ZooKeeper 3.8	3.8.4	zookeeper.apache.org	TGZ	[]
Elasticsearch 8.12	8.12.2	elastic.co	RPM/TGZ	[]
Kibana 8.12	8.12.2	elastic.co	RPM/TGZ	[]
Wazuh Server 4.7	4.7.3	wazuh.com	RPM	[]
Wazuh Indexer 4.7	4.7.3	wazuh.com	RPM	[]
Suricata 7.0	7.0.3	suricata.io	RPM/TGZ	[]
Zeek 6.3	6.3.0	zeek.org	TGZ	[]
Arkime 5.4	5.4.0	arkime.com	TGZ	[]
GRR 3.4	3.4.6.1	github.com	DEB/RPM	[]
Osquery Fleet 4.17	4.17.0	fleetdm.com	DEB/RPM	[]
TheHive 5.4	5.4.4	thehive-project.org	ZIP/TGZ	[]
Cortex 3.1	3.1.6	thehive-project.org	ZIP/TGZ	[]
MISP 2.4	2.4.180	misp-project.org	TGZ	[]
OpenCTI 6.0	6.0.18	opencti.io	DOCKER	[]
OpenVAS/GVM 22.4	22.4.5	greenbone.net	RPM/TGZ	[]
DefectDojo 2.42	2.42.0	defectdojo.org	DOCKER	[]
Prometheus 2.51	2.51.1	prometheus.io	TGZ	[]
Grafana 11.1	11.1.3	grafana.com	RPM/TGZ	[]
Kong Gateway 3.6	3.6.1	konghq.com	DOCKER	[]
Redis 7.2	7.2.4	redis.io	RPM/TGZ	[]
Node.js 20 LTS	20.15.0	nodejs.org	BIN	[]
Python 3.12	3.12.4	python.org	SRC/RPM	[]
Go 1.22	1.22.3	golang.org	BIN/TGZ	[]

Table 1.2: Software Artifacts Inventory

1.3 Network Prerequisites

VLAN Configuration: All seven VLANs (100-700) provisioned on switching infrastructure with appropriate IP subnets assigned per architecture document.

- VLAN 100 (soc-management): 172.16.0.0/16
- VLAN 200 (soc-frontend): 172.28.0.0/16
- VLAN 300 (soc-backend): 172.29.0.0/16
- VLAN 400 (soc-events): 172.30.0.0/16
- VLAN 500 (soc-capture): 172.31.0.0/16
- VLAN 600 (soc-backup): 172.32.0.0/16
- VLAN 700 (soc-monitoring): 172.33.0.0/16

DNS Infrastructure: Internal DNS server(s) configured with forward and reverse lookup zones for all planned hostnames.

- Primary DNS: soc-dns-01.internal.djezzy.dz (172.16.0.10)
- Secondary DNS: soc-dns-02.internal.djezzy.dz (172.16.0.11)
- Domain: internal.djezzy.dz

NTP Time Synchronization: Stratum-2 (or better) NTP servers accessible from management network for accurate timestamp correlation.

- Primary NTP: ntp-01.internal.djezzy.dz (synchronized to GPS or atomic source)
- Backup NTP: ntp-02.internal.djezzy.dz

Firewall Rules: Base firewall policy allowing necessary traffic between zones per security architecture document.

- Default deny all inter-zone traffic
- Explicit allow rules documented in Appendix A

1.4 Personnel & Access Requirements

Role	Minimum Count	Required Skills	Availability
Platform Lead	1	Linux admin, Docker, networking, databases	Full-time Phase 1-8
System Administrator	2	OS installation, hardening, patching	Full-time Phase 1-4
Network Engineer	1	Switching, VLANs, bonding, TCP/IP	Phase 1, 4 (part-time)
Database Administrator	1	PostgreSQL, Kafka, ES administration	Phase 3-5 (part-time)
Security Engineer	2	SIEM, NSM, threat intel tools	Phase 4-6 (full-time)
DevOps Engineer	1	CI/CD, containers, automation	Phase 5-6 (part-time)
Project Manager	1	Coordination, risk mgmt, reporting	Throughout (part-time)

Table 1.3: Personnel Requirements by Phase

1.5 Pre-Deployment Sign-Off

Before proceeding to operating system installation, obtain formal sign-off from the following authorities confirming all prerequisites have been satisfied. Document sign-off with date and signature in project

records.

Authority Role	Name	Signature	Date	Comments
Facility Manager				Power, cooling, space verified
Network Architect				VLANs, DNS, firewall configured
Security Officer				Physical access controls verified
Project Sponsor				Budget, resources approved
Platform Lead				Technical readiness confirmed

Table 1.4: Pre-Deployment Sign-Off Sheet

2. Operating System Installation & Hardening

This section covers base operating system installation across all fourteen primary site servers followed by security hardening according to CIS benchmarks and organizational policies. We recommend Rocky Linux 9.4 (or RHEL 9.4 if subscription licenses are available) as the base operating system due to its enterprise stability, long support lifecycle, and compatibility with container runtimes and database software. Execute these procedures sequentially on each server, documenting any deviations or issues encountered.

2.1 Rocky Linux 9.4 Installation Procedure

Step 1: Create Boot Media

Download the Rocky Linux 9.4 DVD ISO from the official mirror site and verify SHA256 checksum. Create bootable USB media using the dd command or Rufus (Windows). For air-gapped environments, perform this step on an internet-connected workstation then transfer USB drives securely.

```
# Download Rocky Linux 9.4 ISO (on internet-connected machine)
wget https://download.rockylinux.org/pub/rocky/9.4/isos/x86_64/Rocky-9.4-x86_64-dvd.iso

# Verify SHA256 checksum
sha256sum Rocky-9.4-x86_64-dvd.iso

# Expected: d7b1c... (verify against official hash)

# Create bootable USB (replace /dev/sdX with actual device)
dd if=Rocky-9.4-x86_64-dvd.iso of=/dev/sdX bs=4M status=progress conv=fsync
```

Step 2: Install Operating System

- Boot server from USB media (enter BIOS/F2 to select boot device)
- Select 'Install Rocky Linux 9' from boot menu
- Choose language: English (United States)
- Installation Destination: Select all disks, select 'Custom' partitioning scheme
- Create standard partitions:
 - - /boot : 2 GB ext4 xfs (boot flag)
 - - swap : RAM size (min 8GB, max 32GB)
 - - / : 100 GB ext4 xfs (root filesystem)
 - - /var : 200 GB ext4 xfs (logs, transient data)
 - - /var/log : 50 GB ext4 xfs (dedicated log volume)
 - - /data : Remaining ext4/lvm (application data)
- Network & Hostname: Configure hostname per naming convention (see Table 2.1)
- Time Zone: Africa/Algiers, enable NTP
- Software Selection: 'Server with GUI' (minimal) or 'Minimal Install'
- Begin Installation and set root password

Server Role	Hostname Example	IP Address (mgmt)	IP Address (data)
SIEM Master 1	soc-siem-mstr-01.internal.djezzy.dz	172.16.0.101	172.28.0.101
SIEM Master 2	soc-siem-mstr-02.internal.djezzy.dz	172.16.0.102	172.28.0.102
ES Data Node 1	soc-es-data-01.internal.djezzy.dz	172.16.0.111	172.30.0.111

ES Data Node 2	soc-es-data-02.internal.djezzy.dz	172.16.0.112	172.30.0.112
ES Data Node 3	soc-es-data-03.internal.djezzy.dz	172.16.0.113	172.30.0.113
ES Data Node 4	soc-es-data-04.internal.djezzy.dz	172.16.0.114	172.30.0.114
DB Primary	soc-db-primary.internal.djezzy.dz	172.16.0.121	172.30.0.121
DB Replica 1	soc-db-replica-01.internal.djezzy.dz	172.16.0.122	172.30.0.122
DB Replica 2	soc-db-replica-02.internal.djezzy.dz	172.16.0.123	172.30.0.123
NSM Capture 1	soc-nsm-capture-01.internal.djezzy.dz	172.16.0.131	172.31.0.131
NSM Capture 2	soc-nsm-capture-02.internal.djezzy.dz	172.16.0.132	172.31.0.132
App Server 1	soc-app-01.internal.djezzy.dz	172.16.0.141	172.28.0.141
App Server 2	soc-app-02.internal.djezzy.dz	172.16.0.142	172.28.0.142
Archive Server	soc-archive-01.internal.djezzy.dz	172.16.0.151	172.32.0.151

Table 2.1: Server Naming Convention and IP Assignment

2.2 Post-Installation Base Configuration

Step 3: Initial Network Configuration

After first boot, configure network interfaces with static IPs according to the addressing scheme above. Each server requires minimum two interfaces: one for management (VLAN 100) and one or more for data traffic depending on role. Use NetworkManager nmcli commands for consistent configuration across all nodes.

```
# View available network interfaces
nmcli device status

# Configure management interface (example for SIEM Master 1)
nmcli con add type ethernet ifname enp1s0f0 con-name mgmt \
ip4 172.16.0.101/16 \
gw4 172.16.0.1 \
ipv4.dns "172.16.0.10 172.16.0.11"

# Activate connection
nmcli con up mgmt

# Verify connectivity
ping -c 4 172.16.0.1 # Default gateway
ping -c 4 172.16.0.10 # DNS server
ping -c 4 google.com # External (if allowed)
```

Step 4: System Updates and Essential Packages

```
# Update all packages to latest versions
dnf update -y && dnf upgrade --refresh -y

# Install essential utilities
dnf install -y \
```

```
vim-enhanced htop iotop net-tools bind-utils \  
wget curl git rsync lsof strace tcpdump \  
chrony logrotate fail2ban audit \  
epel-release  
  
# Enable EPEL repository for additional packages  
dnf install -y epel-release
```

2.3 Security Hardening (CIS Benchmark)

Apply CIS Benchmark Level 1 hardening recommendations for Rocky Linux 9. The following configurations address critical security controls including file system permissions, service hardening, network configuration, logging, and access control. Execute these commands on every server after base OS installation completes.

Step 5: File System Security

```
# Set restrictive umask globally  
echo 'umask 027' >> /etc/profile  
echo 'umask 027' >> /etc/bashrc  
  
# Secure /tmp with nosuid,noexec,nodev  
tmpfs /tmp tmpfs defaults,rw,nosuid,nodev,noexec,size=4G 0 0 >> /etc/fstab  
  
# Set proper permissions on sensitive files  
chmod 600 /etc/shadow /etc/gshadow  
chmod 644 /etc/passwd /etc/group  
chown root:root /etc/passwd /etc/shadow /etc/group /etc/gshadow
```

Step 6: SSH Hardening

```
# Backup original config  
cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup  
  
# Apply hardened SSH configuration  
cat > /etc/ssh/sshd_config.d/hardened.conf << 'EOF'  
# SSH Hardening - CIS Benchmark Level 1  
Protocol 2  
PermitRootLogin prohibit-password  
MaxAuthTries 3  
LoginGraceTime 30  
X11Forwarding no  
AllowTcpForwarding no  
PermitEmptyPasswords no  
ChallengeResponseAuthentication no  
UseDNS yes  
AllowUsers  
PasswordAuthentication no # Key-only auth recommended  
PubkeyAuthentication yes  
AuthorizedKeysFile .ssh/authorized_keys
```

```
LogLevel VERBOSE
Banner /etc/issue.net
EOF

# Restart SSH service
systemctl restart sshd
```

Step 7: Firewall Configuration

```
# Enable and start firewalld
systemctl enable --now firewalld

# Set default zone to drop (deny all)
firewall-cmd --set-default-zone=drop

# Allow management network (adjust as needed)
firewall-cmd --permanent --zone=trusted \
--add-source=172.16.0.0/16

# Allow specific services if required
firewall-cmd --permanent --zone=trusted --add-service=ssh
firewall-cmd --permanent --zone=trusted --add-service=https

# Reload firewall rules
firewall-cmd --reload
```

Step 8: Kernel Parameters and Limits

```
# Set kernel parameters for security and performance
cat > /etc/sysctl.d/99-security-hardening.conf << 'EOF'

# Network Security
net.ipv4.ip_forward = 0
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.default.send_redirects = 0
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.default.accept_redirects = 0
net.ipv4.icmp_echo_ignore_broadcasts = 1
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1

# SYN flood protection
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_max_syn_backlog = 2048
net.ipv4.tcp_synack_retries = 2

# Performance tuning for databases/ES
vm.swappiness = 1
vm.dirty_ratio = 15
vm.dirty_background_ratio = 5
fs.file-max = 2097152
```

```
net.core.somaxconn = 65535
net.ipv4.tcp_tw_reuse = 1
net.ipv4.ip_local_port_range = 1024 65535
EOF

# Apply kernel parameters
sysctl -p /etc/sysctl.d/99-security-hardening.conf

# Set file descriptor limits for high-concurrency services
cat > /etc/security/limits.d/99-soc-platform.conf << 'EOF'
* soft nofile 1048576
* hard nofile 1048576
* soft nproc 65535
* hard nproc 65535
* soft memlock unlimited
* hard memlock unlimited
root soft nofile 1048576
root hard nofile 1048576
EOF
```

2.4 Validation Checklist

- ☐ OS boots successfully without errors
- ☐ All network interfaces configured with correct IPs
- ☐ Ping gateway, DNS, and peer servers successful
- ☐ SSH login works with key-based authentication only
- ☐ Firewall rules permit expected traffic, block others
- ☐ Kernel parameters applied correctly (sysctl -a | grep)
- ☐ File descriptor limits visible (ulimit -a)
- ☐ Time synchronized via NTP (timedatectl status)
- ☐ SELinux in enforcing mode (sestatus)
- ☐ No unnecessary services running (systemctl list-units)

3. Container Runtime & Orchestration Setup

The Djezzy SOC Platform deploys all fifteen security tools plus supporting infrastructure services as Docker containers orchestrated via Docker Compose. This section covers Docker Community Edition installation, daemon configuration optimized for production workloads, and Docker Compose installation for multi-container orchestration. Container runtime must be installed identically on all fourteen servers to ensure consistent behavior during service scheduling.

3.1 Docker CE Installation

Step 1: Add Docker Repository

```
# Install dependencies
dnf install -y yum-utils device-mapper-persistent-data lvm2

# Add Docker CE stable repository
yum-config-manager --add-repo \
https://download.docker.com/linux/centos/docker-ce.repo

# For air-gapped: Download RPMs separately and transfer via USB
# Download from: https://download.docker.com/linux/centos/9/x86_64/stable/Packages/
```

Step 2: Install Docker Components

```
# Install Docker CE, CLI, Compose plugin, and containerd
dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Start and enable Docker service
systemctl enable --now docker

# Add platform engineers to docker group
usermod -aG docker

# Verify installation
docker version
docker compose version
docker run --rm hello-world
```

3.2 Production Daemon Configuration

Configure Docker daemon with settings optimized for enterprise security workloads including storage driver selection, logging configuration, resource limits, and security options. These settings apply to all containers launched on this host.

```
# Backup existing config
cp /etc/docker/daemon.json /etc/docker/daemon.json.backup 2>/dev/null || true

# Create production Docker daemon configuration
cat > /etc/docker/daemon.json << 'DAEMON_EOF'
{
  "log-driver": "json-file",
  "log-opts": {
```

```

"max-size": "100m",
"max-file": "10"
},
"storage-driver": "overlay2",
"storage-opts": [
"overlay2.override_kernel_check=true"
],
"default-ulimits": {
"nofile": {
"Name": "nofile",
"Hard": 1048576,
"Soft": 1048576
}
},
"max-concurrent-downloads": 10,
"max-concurrent-uploads": 10,
"live-restore": true,
"userland-proxy": false,
"init": true,
"iptables": true,
"ip6tables": true,
"bridge": "none",
"ip": "none",
"insecure-registries": [],
"registry-mirrors": []
}
DAEMON_EOF

# Restart Docker to apply changes
systemctl restart docker

systemctl status docker

```

3.3 Custom Bridge Networks

Create the four isolated bridge networks defined in the architecture document. These networks provide Layer 2 isolation between service tiers and map to physical VLANs where required for inter-server communication.

```

# Create SOC platform networks
# Frontend network (user-facing apps, API gateway)
docker network create \
--driver bridge \
--subnet 172.28.0.0/16 \
--gateway 172.28.0.1 \
--opt com.docker.network.bridge.name=soc_frontend \
soc-frontend

```

```

# Backend network (SOAR, Threat Intel, Vuln)
docker network create \
--driver bridge \
--subnet 172.29.0.0/16 \
--gateway 172.29.0.1 \
--opt com.docker.network.bridge.name=soc_backend \
soc-backend

# Events network (SIEM, DB, Kafka, ES)
docker network create \
--driver bridge \
--subnet 172.30.0.0/16 \
--gateway 172.30.0.1 \
--opt com.docker.network.bridge.name=soc_events \
soc-events

# Monitoring network (Prometheus, Grafana)
docker network create \
--driver bridge \
--subnet 172.31.0.0/16 \
--gateway 172.31.0.1 \
--opt com.docker.network.bridge.name=soc_monitoring \
soc-monitoring

# Verify networks created
docker network ls | grep soc-

docker network inspect soc-frontend

```

3.4 Validation

- [] Docker daemon running without errors (systemctl status docker)
- [] Docker version matches target (docker version >= 27.0)
- [] Docker Compose plugin working (docker compose version)
- [] Container can run hello-world successfully

```
[ ] All four SOC networks exist (docker network ls)
```

- [] Network subnets match specification (docker network inspect)
- [] Log rotation configured (check /var/log/docker/)
- [] Storage driver is overlay2 (docker info | grep Storage)

4. Network Configuration & Segmentation

This section details advanced network configuration including interface bonding for redundancy, VLAN tagging for segmentation, and optimization settings for high-throughput workloads. Proper network configuration is critical for the NSM capture nodes that must sustain 100Gbps line-rate packet processing without loss.

4.1 Bonded Interface Configuration (LACP)

For servers requiring high availability and increased bandwidth, configure bonded interfaces using LACP (802.3ad) protocol. This applies particularly to Elasticsearch data nodes, database cluster members, and application servers with multiple data interfaces.

```
# Example: Bond configuration for ES Data Node (2x 25GbE)
# Create bond interface file
cat > /etc/sysconfig/network-scripts/ifcfg-bond0 << 'EOF'
DEVICE=bond0
TYPE=Bond
BONDING_MASTER=yes
IPADDR=172.30.0.111
PREFIX=16
GATEWAY=172.30.0.1
DNS1=172.16.0.10
BONDING_MODE=4 # LACP (802.3ad)
BONDING_XMIT_HASH_POLICY=layer3+4
MIIMON=100 # Link monitoring interval (ms)
LACP_RATE=fast # LACPDU transmission rate
EOF

# Configure slave interfaces
cat > /etc/sysconfig/network-scripts/ifcfg-enp1s0f0 << 'EOF'
DEVICE=enp1s0f0
TYPE=Ethernet
MASTER=bond0
SLAVE=yes
BOOTPROTO=none
EOF

# Repeat for enp1s0f1 (second slave)

# Activate bond
ifdown enp1s0f0 enp1s0f1 && ifup bond0

# Verify bond status
cat /proc/net/bonding/bond0
ip link show bond0
```

4.2 NSM Capture Node Network Optimization

Packet capture nodes require special network tuning to achieve line-rate capture at 100Gbps without packet loss. These optimizations include increasing ring buffer sizes, enabling multi-queue RSS, and configuring interrupt affinity for optimal CPU utilization.

```
# Increase NIC ring buffers for high-throughput capture
# (Requires ethtool and NIC driver support)

# Set RX/TX ring buffer to maximum
ethtool -G enp134s0 rx 8192 tx 8192

# Enable multi-queue RSS (Receive Side Scaling)
ethtool -L enp134s0 combined 16

# Set interrupt coalescing for throughput vs latency tradeoff
ethtool -C enp134s0 adaptive-rx on adaptive-tx on
ethtool -C enp134s0 rx-usecs 50 tx-usecs 50

# Make settings persistent across reboots
cat > /etc/udev/rules.d/99-nic-tuning.rules << 'EOF'
ACTION=="add", SUBSYSTEM=="net", KERNELS=="enp134s0", \
RUN+="/sbin/ethtool -G $kernel rx 8192 tx 8192"
ACTION=="add", SUBSYSTEM=="net", KERNELS=="enp134s0", \
RUN+="/sbin/ethtool -L $kernel combined 16"
EOF
```

4.3 DNS Configuration

Configure local DNS resolution for all SOC platform hostnames. This ensures consistent name resolution across all services regardless of which network zone they reside in. Use the internal DNS servers configured during pre-deployment.

```
# Configure /etc/resolv.conf (via NetworkManager or direct)
cat > /etc/resolv.conf << 'EOF'
search internal.djezzy.dz
nameserver 172.16.0.10
nameserver 172.16.0.11
options rotate
options timeout:2
options attempts:3
EOF

# Test DNS resolution
dig soc-es-data-01.internal.djezzy.dz +short
dig soc-db-primary.internal.djezzy.dz +short
nslookup soc-nsm-capture-01.internal.djezzy.dz
```

5. Database Layer Deployment

The database layer forms the persistent storage foundation for the entire SOC platform. This section covers PostgreSQL 16 for structured data (cases, findings, configuration), Apache Kafka 3.6 for event streaming, Redis 7 for caching, and ZooKeeper for Kafka coordination. Deploy these components on the three dedicated database cluster nodes (soc-db-primary, soc-db-replica-01, soc-db-replica-02) following the procedures below.

5.1 PostgreSQL 16 Installation

Step 1: Add PostgreSQL Repository

```
# Install PostgreSQL 16 repository
dnf install -y
https://download.postgresql.org/pub/repos/yum/reporepms/EL-9-x86_64/pgdg-redhat-repo-latest.noarch.rpm
dnf install -y postgresql16-server postgresql16 contrib

# Initialize database cluster
/usr/pgsql-16/bin/postgresql-16-setup initdb
```

Step 2: Configure PostgreSQL for High Performance

```
# Edit postgresql.conf for SOC workload optimization
vi /var/lib/pgsql/16/data/postgresql.conf

# Key performance settings (add/modify in postgresql.conf):

# Connection settings
listen_addresses = '*'
port = 5432
max_connections = 500

# Memory settings (tune based on server RAM - this example for 384GB)
shared_buffers = 96GB # 25% of RAM
effective_cache_size = 288GB # 75% of RAM
work_mem = 256MB # Per-sort memory
maintenance_work_mem = 2GB # Maintenance ops
huge_pages = try # Use huge pages

# WAL settings for replication
wal_level = replica
max_wal_senders = 5
wal_keep_size = 32GB
wal_compression = zstd

# Query optimizer
random_page_cost = 1.1 # SSD optimization
effective_io_concurrency = 200 # SSD parallel I/O
default_statistics_target = 200
```

Step 3: Configure Streaming Replication

```
# On PRIMARY server - create replication user
```

```

sudo -u postgres psql -c "CREATE USER replicator WITH REPLICATION ENCRYPTED PASSWORD '');"

# Edit pg_hba.conf to allow replication connections
echo 'host replication replicator 172.30.0.0/16 scram-sha-256' >> /var/lib/pgsql/16/data/pg_hba.conf
echo 'host all all 172.30.0.0/16 scram-sha-256' >> /var/lib/pgsql/16/data/pg_hba.conf

# Restart PostgreSQL
systemctl restart postgresql-16
systemctl enable postgresql-16

# On REPLICA servers - base backup from primary
# Stop PostgreSQL on replica first
systemctl stop postgresql-16
rm -rf /var/lib/pgsql/16/data/*

# Perform base backup from primary
sudo -u postgres /usr/pgsql-16/bin/pg_basebackup \
-h soc-db-primary.internal.djezzy.dz \
-U replicator \
-D /var/lib/pgsql/16/data \
-Fp -Xs -P -R

# Create standby.signal file on replica
sudo -u postgres touch /var/lib/pgsql/16/data/standby.signal

# Configure primary_conninfo in postgresql.auto.conf
echo "primary_conninfo = 'host=soc-db-primary.internal.djezzy.dz port=5432 user=replicator password='" \
> /var/lib/pgsql/16/data/postgresql.auto.conf

# Start replica - it will connect to primary and begin replicating
chown -R postgres:postgres /var/lib/pgsql/16/data
systemctl start postgresql-16

```

5.2 Apache Kafka 3.6 Installation

```

# Download and extract Kafka
cd /opt
wget https://archive.apache.org/dist/kafka/3.6.1/kafka_2.13-3.6.1.tgz
tar -xzf kafka_2.13-3.6.1.tgz
ln -s kafka_2.13-3.6.1 kafka
useradd -r -m -d /opt/kafka kafka
chown -R kafka:kafka /opt/kafka*

# Create Kafka data directories
mkdir -p /data/kafka/logs /data/kafka/data
chown -R kafka:kafka /data/kafka

# Configure ZooKeeper (on each broker node)
cat > /opt/kafka/config/zookeeper.properties << 'EOF'
dataDir=/data/kafka/zookeeper

```

```

clientPort=2181
maxClientCnxns=200
tickTime=2000
initLimit=10
syncLimit=5
# Server IDs for 3-node ensemble:
server.1=soc-db-primary:2888:3888
server.2=soc-db-replica-01:2888:3888
server.3=soc-db-replica-02:2888:3888
autopurge.purgeInterval=24
autopurge.snapRetainCount=3
EOF

# Create myid file (unique per node: 1, 2, or 3)
echo "1" > /data/kafka/zookeeper/myid # On primary

# Configure Kafka Broker
cat > /opt/kafka/config/server.properties << 'EOF'
broker.id=1 # Unique per broker
listeners=PLAINTEXT://172.30.0.121:9092
advertised.listeners=PLAINTEXT://soc-db-primary.internal.djezzy.dz:9092
num.network.threads=8
num.io.threads=16
socket.send.buffer.bytes=1024000
socket.receive.buffer.bytes=1024000
socket.request.max.bytes=104857600
log.dirs=/data/kafka/data
num.partitions=32 # Default partition count
num.recovery.threads.per.data.dir=4
offsets.topic.replication.factor=3
transaction.state.log.replication.factor=3
transaction.state.log.min.isr=2
log.retention.hours=336 # 14 days retention
log.segment.bytes=1073741824 # 1GB segment size
log.retention.check.interval.ms=300000
zookeeper.connect=soc-db-primary:2181,soc-db-replica-01:2181,soc-db-replica-02:2181
zookeeper.connection.timeout.ms=18000
group.initial.rebalance.delay.ms=0
delete.topic.enable=true
auto.create.topics.enable=false
compression.type=lz4
EOF

```

5.3 Redis 7 Cache Layer

```
# Install Redis 7
```

```

dnf module enable redis:7 -y
dnf install redis -y

# Configure Redis for production use
cat > /etc/redis/redis.conf << 'EOF'
bind 172.30.0.121 127.0.0.1 # Bind to events network + localhost
port 6379
protected-mode yes
requirepass
maxmemory 64gb # Limit to 64GB of 128GB RAM
maxmemory-policy allkeys-lru
save 900 1 # Snapshot every 15 min if 1 key changed
save 300 10
save 60 10000
appendonly yes # AOF persistence
appendfilename "appendonly.aof"
appendfsync everysec
tcp-backlog 511
timeout 0
tcp-keepalive 300
loglevel notice
databases 16
EOF

# Enable and start Redis
systemctl enable --now redis

```

5.4 PgBouncer Connection Pooler

PgBouncer sits between application containers and PostgreSQL, managing connection pools efficiently for the microservices architecture that generates thousands of concurrent connections. Install PgBouncer on all three database nodes.

```

# Install PgBouncer
dnf install -y pgbouncer

# Configure PgBouncer
cat > /etc/pgbouncer/pgbouncer.ini << 'EOF'
[databases]
soc_platform = host=172.0.0.1 port=5432 dbname=socdb

[pgbouncer]
pool_mode = transaction
listen_addr = 172.30.0.121
listen_port = 6432
auth_type = md5
auth_file = /etc/pgbouncer/userlist.txt
admin_users = postgres

```

```
default_pool_size = 25
max_client_conn = 5000
reserve_pool_size = 5
reserve_pool_timeout = 3
server_reset_query = DISCARD ALL
server_check_query = SELECT 1
server_check_delay = 30
log_connections = 1
log_disconnections = 1
log_pooler_errors = 1
stats_period = 60
EOF

# Create user list for authentication
echo '"soc_app" ""' > /etc/pgbouncer/userlist.txt
chmod 640 /etc/pgbouncer/userlist.txt
chown pgbouncer:pgbouncer /etc/pgbouncer/userlist.txt

# Enable and start PgBouncer
systemctl enable --now pgbouncer
```

6. Elasticsearch Cluster Deployment

Elasticsearch serves as the primary search and analytics engine for the SOC platform, indexing security events from Wazuh, logs from all services, and providing the query backend for Kibana dashboards and investigations. This section covers Elasticsearch 8.12 cluster deployment across four dedicated data nodes plus master-eligible nodes on the SIEM masters.

6.1 Elasticsearch Installation

```
# Import Elasticsearch GPG key and repository
rpm --import https://artifacts.elastic.co/GPG-KEY-elasticsearch

cat > /etc/yum.repos.d/elasticsearch.repo << 'EOF'
[elasticsearch-8.x]
name=Elasticsearch repository for 8.x packages
baseurl=https://artifacts.elastic.co/packages/8.x/yum
gpgcheck=1
gpgkey=https://artifacts.elastic.co/GPG-KEY-elasticsearch
enabled=1
autorefresh=1
type=rpm-md
EOF

# Install Elasticsearch
dnf install -y elasticsearch
```

6.2 JVM and Memory Configuration

Elasticsearch runs on Java Virtual Machine (JVM) and requires careful memory tuning. Allocate approximately 50% of system RAM to Elasticsearch heap, leaving the remainder for operating system filesystem cache that significantly improves index segment access performance.

```
# Configure JVM options (for 512GB RAM server -> ~256GB heap)
cat > /etc/elasticsearch/jvm.options.d/soc-platform.options << 'EOF'
-Xms256g
-Xmx256g

# GC settings for large heaps
-XX:+UseG1GC
-XX:G1HeapRegionSize=16m
-XX:InitiatingHeapOccupancyPercent=30
-XX:G1ReservePercent=15

# Disable DNS caching (use system resolver)
-Des.networkaddress.cache.ttl=10
-Des.networkaddress.cache.negative.ttl=5
EOF
```

6.3 Elasticsearch Cluster Configuration

```

# Main elasticsearch.yml configuration
cat > /etc/elasticsearch/elasticsearch.yml << 'EOF'

cluster.name: djezzy-soc-cluster


# Node identity
node.name: es-data-01 # Unique per node
node.roles: [data, data_hot, data_warm, ingest]


# Network binding
network.host: 172.30.0.111 # Events network IP
http.port: 9200
transport.port: 9300


# Discovery - seed hosts are master-eligible nodes
discovery.seed_hosts: ["172.30.0.101", "172.30.0.102"]
cluster.initial_master_nodes: ["es-master-01", "es-master-02"]


# Security (initial setup generates passwords)
xpack.security.enabled: true
xpack.security.enrollment.enabled: false
xpack.security.http.ssl.enabled: false # Enable for production!
xpack.security.transport.ssl.enabled: true
xpack.security.transport.ssl.verification_mode: certificate
xpack.security.transport.ssl.keystore.path: elastic-certificates.p12
xpack.security.transport.ssl.truststore.path: elastic-certificates.p12


# Paths
path.data: /data/elasticsearch/data
path.logs: /var/log/elasticsearch


# Memory locking (critical for performance)
bootstrap.memory_lock: true


# Performance tuning
indices.memory.index_buffer_size: 30%
indices.query.cache.size: 15%
thread_pool.search.size: 20
thread_pool.write.size: 20
thread_pool.get.size: 20


# Index lifecycle (ILM)
indices.lifecycle.poll_interval: 1m


# Snapshot repository (local)
path.repo: ["/data/elasticsearch/snapshots"]

EOF

```

6.4 Starting and Validating Cluster

```
# Enable memory locking for elasticsearch user
```



```
systemctl edit elasticsearch
# Add under [Service]:
# LimitMEMLOCK=infinity
# LimitNOFILE=1048576
# LimitNPROC=65535

# Start Elasticsearch
systemctl daemon-reload
systemctl enable --now elasticsearch

# Wait for cluster to form (~30 seconds)
sleep 30

# Check cluster health
curl -X GET "localhost:9200/_cluster/health?pretty" -u elastic:
# Expected response: "status": "green" or "yellow"

# Check nodes
curl -X GET "localhost:9200/_cat/nodes?v" -u elastic:

# Reset built-in passwords (first time only)
/usr/share/elasticsearch/bin/elasticsearch-reset-password -i -u elastic
/usr/share/elasticsearch/bin/elasticsearch-reset-password -i -u kibana_system
/usr/share/elasticsearch/bin/elasticsearch-reset-password -i -u logstash_system
```

7-14. Additional Deployment Sections

Due to document length constraints, sections 7 through 14 follow the same detailed format as sections 1-6 above. Below is a summary of what each section covers. The complete implementation guide includes full command sequences, configuration templates, and validation procedures for each component.

Section 7: SIEM Stack (Wazuh/Elasticsearch/Kibana)

- Wazuh manager installation and agent registration
- Wazuh indexer configuration for Elasticsearch backend
- Kibana deployment with SOC-specific dashboards
- Integration between Wazuh, Elasticsearch, and Kibana
- Alert rule configuration and notification setup

Section 8: Network Security Monitoring (Suricata/Zeek/Arkime)

- Suricata IDS/IPS installation with PF_RING/AF_XDP support
- Rule set management (Emerging Threats, Proofpoint, custom)
- Zeek/Bro deployment with SOC-specific scripts
- Arkime/Moloch PCAP viewer installation and configuration
- Integration pipeline: Suricata EVE -> Kafka -> ES -> Kibana

Section 9: Endpoint Detection & Response (GRR/Osquery)

- GRR Rapid Response server deployment
- Osquery Fleet server installation and enrollment
- Endpoint agent packaging and distribution
- Hunt workflow creation and execution
- Forensic artifact collection procedures

Section 10: SOAR Platform (TheHive/Cortex)

- TheHive case management installation
- Cortex analyzer/responder configuration
- Analyzer integrations (VirusTotal, MISP, OpenCTI, etc.)
- Case template customization for telecom use cases
- Workflow automation and playbook development

Section 11: Threat Intelligence (MISP/OpenCTI)

- MISP threat sharing platform deployment
- OpenCTI intelligence management installation
- Feed configuration (external and internal sources)
- IOC enrichment pipeline setup
- Bi-directional MISP-TheHive integration

Section 12: Vulnerability Management (OpenVAS/DefectDojo)

- OpenVAS/GVM scanner installation and configuration
- Scan schedule and target definition
- DefectDojo vulnerability correlation platform
- Import pipeline: OpenVAS -> DefectDojo -> TheHive
- Report generation and metric dashboards

Section 13: Monitoring & Observability (Prometheus/Grafana)

- Prometheus metrics collection deployment
- Node exporter, cAdvisor, Blackbox exporter setup
- Grafana visualization platform installation
- SOC-specific dashboard creation (infrastructure, security, business)
- AlertManager notification routing (email, Slack, PagerDuty)

Section 14: API Gateway & Frontend Application

- Kong API Gateway deployment and plugin configuration
- Next.js 16 frontend application build and deployment
- TypeScript backend API services startup
- Authentication framework activation (LDAP/local)
- SSL certificate installation and HTTPS enforcement

15. Integration Testing & Validation

After completing individual component deployment, execute comprehensive integration testing to validate end-to-end data flows, cross-tool communication, and system behavior under realistic load conditions. This section defines test scenarios, acceptance criteria, and troubleshooting procedures for common integration issues.

15.1 Smoke Tests (Basic Functionality)

Test ID	Test Scenario	Command/Procedure	Expected Result	Status
SM-001	PostgreSQL reachable	psql -h soc-db-primary -U soc_app -c 'SELECT 1'	Connected to PostgreSQL. Returns 1	[]
SM-002	Kafka broker responsive	kafka-topics.sh --bootstrap-server soc-app-01:9092 --zookeeper soc-app-01:2181 --list	Topic list (empty) exists	[]
SM-003	Redis accepts connections	redis-cli -h soc-db-primary -a <password> PING	Response: PONG	[]
SM-004	ES cluster healthy	curl -s soc-es-data-01:9200/_cluster/health?pretty	"status": "green" or "yellow"	[]
SM-005	Kibana dashboard load	Browser: https://soc-siem-mstr-01:5601	Page renders	[]
SM-006	Wazuh manager active	curl -s soc-siem-mstr-01:55000/manager/status	{"data": "running"}	[]
SM-007	Suricata running	suricatactl status	Process active, socket listening	[]
SM-008	TheHive accessible	curl -s soc-app-01:9000/api/health	{"status": "ok" ...}	[]
SM-009	Cortex responds	curl -s soc-app-01:9900/cortex/status	JSON status response	[]
SM-010	Grafana dashboards load	Browser: https://soc-app-01:3000	Dashboard list visible	[]
SM-011	Prometheus scraping	curl -s soc-app-01:9090/api/v1/targets	Targets with UP state	[]
SM-012	Kong API gateway	curl -s soc-app-01:8000/status	{"version": "..."}	[]

Table 15.1: Integration Smoke Test Matrix

15.2 End-to-End Event Flow Test

1. Generate test security event (send syslog message to Wazuh or use Filebeat)
2. Verify Wazuh receives and parses event (check Wazuh alerts JSON output)
3. Confirm event indexed in Elasticsearch (query recent indices for test event)
4. Validate Kibana displays event in Discover view within 30 seconds
5. Check alert triggered if rule matches test event pattern
6. Verify TheHive can receive alert via webhook (create test case from alert)
7. Confirm Cortex analyzer executes on case observables (run MISP lookup)
8. Review Grafana dashboard shows incremented event count

15.3 Load Test Scenarios

Scenario	Duration	Target Load	Validation Metrics	Pass Criteria
Event Ingestion	4 hours	50,000 EPS sustained	Kafka lag, ES indexing latency	Lag < 1000 msgs, latency < 5s
Concurrent Queries	2 hours	50 analysts active	Query response time, ES CPU	QRT < 3s, CPU < 80%
PCAP Capture	1 hour	50Gbps sustained	Suricata drops, disk I/O	Drop rate < 0.01%, IOPS headroom

Alert Generation	4 hours	High-alert rule dataset	Alert queue depth, TheHighQueue	Queue < 10K, API latency < 500ms
Full Platform	24 hours	Production-like mixed workload	All components healthy	No critical alerts, zero data loss

Table 15.2: Load Test Scenarios and Acceptance Criteria

16. Operational Runbooks & Procedures

This section provides standardized operating procedures for common day-to-day and emergency operations tasks. Each runbook includes purpose, prerequisites, step-by-step procedures, validation steps, rollback instructions, and escalation criteria. Train all operations personnel on these procedures and conduct quarterly drills to maintain proficiency.

16.1 Daily Operations Checklist

Time	Task	Command/Action	Expected Result	Escalate If
08:00	Cluster health review	Check ES health, PG replication	All nodes healthy	Any red/unhealthy
08:30	Disk capacity check	df -h grep -E "/data /var/log"	> 80% triggers action	< 85% used
09:00	Alert queue review	TheHive alert count, unresolved	No backlog > 24hrs	Backlog growing
10:00	Backup verification	Check overnight backup jobs complete	Successful status all jobs	Any failure
12:00	Performance spot-check	Query latencies, resource utilization	Within SLA thresholds	Repeated SLA miss
14:00	Security review	New findings, IOC hits, anomalies	Normal baseline patterns	Significant deviation
16:00	Log review for errors	grep ERROR /var/log/*/{es,kafka,grafana}	Zero critical errors	Repeating errors
17:00	End-of-day summary	Compile daily metrics, update shift log	Shift log complete	Missing entries

Table 16.1: Daily Operations Checklist

16.2 Runbook: Elasticsearch Node Failure Recovery

Purpose: Recover Elasticsearch cluster operation following single data node failure.

Impact: Reduced search capacity, potential shard relocation causing elevated cluster load.

RTO Target: < 4 hours to restore full capacity.

Procedure:

- IDENTIFY: Determine failed node via curl 'localhost:9200/_cat/nodes?v' (missing entry) or monitoring alert
- ASSESS: Check if node is recoverable (hardware issue vs. OS/service crash) via IPMI/console access
- ATTEMPT RESTART: If hardware OK, attempt service restart: systemctl restart elasticsearch
- MONITOR: Watch shard recovery progress: curl 'localhost:9200/_cat/recovery?v&active;_only=true'
- If recovery stalls > 1 hour, proceed to node replacement procedure
- REPLACE (if needed): Provision new server, join to cluster, allow shard rebalancing
- VALIDATE: Confirm cluster returns to green state: curl 'localhost:9200/_cluster/health?pretty'
- DOCUMENT: Record incident details, root cause, and prevention measures

16.3 Runbook: PostgreSQL Failover to Replica

Purpose: Promote standby replica to primary role when current primary fails.

Prerequisites: At least one healthy replica with minimal replication lag (< 5 minutes).

```
# Step 1: Verify replica status (on healthy replica server)
sudo -u postgres psql -c "SELECT * FROM pg_stat_replication;"

# Step 2: Check replication lag (should be < 5 minutes ideally)
```

```
sudo -u postgres psql -c "SELECT now() - pg_last_xact_replay_timestamp() AS replication_lag;"
```

```
# Step 3: If primary is truly down, promote replica
```

```
sudo -u postgres /usr/pgsql-16/bin/pg_ctl promote -D /var/lib/pgsql/16/data
```

```
# Step 4: Update application connection strings / DNS to point to new primary
```

```
# Update PgBouncer target, Kafka Connect sinks, application configs
```

```
# Step 5: Verify applications connecting successfully
```

```
psql -h -U soc_app -d socdb -c "SELECT 1;"
```

```
# Step 6: Rebuild old primary as new replica once recovered
```

```
# Follow Section 5.1 Step 3 base backup procedure
```

17. Troubleshooting Guide

This section catalogs common issues encountered during SOC platform operation along with diagnostic procedures and resolution steps. Use this guide as first-line troubleshooting reference before escalating to vendors or senior engineers. Each entry includes symptoms, likely causes, diagnostic commands, and proven solutions.

17.1 Elasticsearch Issues

Symptom: Cluster stuck in RED state

Likely Causes: Unassigned shards due to node failure, allocation settings, or disk watermark

Diagnosis: GET _cluster/allocation/explain, GET _cat/shards?v&s;=state

Resolution: Check disk watermarks, adjust allocation settings, add nodes or increase disk

Symptom: Slow search queries (> 10 seconds)

Likely Causes: Insufficient heap, field data cache saturation, expensive aggregations, missing index optimization

Diagnosis: GET _nodes/stats, GET _cache/stats, slow query log analysis

Resolution: Increase heap, optimize mappings, review query patterns, consider index design

Symptom: OutOfMemoryError in Elasticsearch logs

Likely Causes: JVM heap too small, field data explosion, aggregation memory limits

Diagnosis: Check jvm.options -Xmx setting, _nodes/stats jvm, GC log frequency

Resolution: Increase heap (max 50% RAM), tune circuit_breaker limits, optimize queries

17.2 Kafka Issues

Symptom: Consumer lag increasing continuously

Resolution: Add consumer instances, rebalance partitions, scale brokers

Symptom: Under-replicated partitions

Resolution: Restart failed broker, replace failed disk, increase replication factor

Symptom: ZooKeeper session expired errors

Resolution: Reduce Kafka GC pressure, check network, scale ZK ensemble

17.3 PostgreSQL Issues

Symptom: Replication lag growing beyond acceptable

Resolution: Check network, upgrade replica hardware, reduce write load or batch commits

Symptom: Connection pool exhaustion (PgBouncer errors)

Resolution: Increase max_client_conn/default_pool_size, fix app connection leaks

Symptom: Vacuum not keeping up (table bloat)

Resolution: Tune autovacuum params, run manual VACUUM ANALYZE, kill blocking transactions

17.4 Docker/Container Issues

Symptom: Container OOM killed (exit code 137)

Resolution: Increase `deploy.resources.limits.memory`, investigate leak, add host RAM

Symptom: Containers cannot reach each other (cross-network)

Resolution: Attach to correct network (`--network soc-events`), check firewall rules

Symptom: Docker daemon unresponsive, high CPU

Resolution: Restart daemon, prune unused images/networks, reduce log retention

Appendix A: Configuration Reference

This appendix provides quick-reference configuration templates for all major components. Use these as starting points, adjusting values based on your specific hardware specifications and workload characteristics.

A.1 Environment Variables Summary

Variable	Component	Example Value	Description
POSTGRES_HOST	All apps	soc-db-primary	PostgreSQL primary hostname
POSTGRES_PORT	All apps	5432	PgBouncer port (not direct PG)
POSTGRES_DB	All apps	socdb	Application database name
POSTGRES_USER	All apps	soc_app	Application DB user
POSTGRES_PASSWORD	All apps	<secure>	From secrets management
KAFKA_BOOTSTRAP_SERVERS	Producers/Consumers	soc-db-primary:9092,soc-db-replica-01:9092	Kafka broker list
ES_HOSTS	Wazuh/Kibana	https://es-data-01:9200,https://es-data-02:9200	ES cluster endpoints
ELASTIC_PASSWORD	Wazuh/Kibana	<secure>	ES built-in user password
REDIS_HOST	Session cache	soc-db-primary	Redis server hostname
REDIS_PORT	Session cache	6379	Redis port
THEHIVE_URL	Cortex	http://soc-app-01:9000	TheHive API endpoint
CORTEX_URL	TheHive	http://soc-app-01:9900	Cortex API endpoint
MISP_URL	TheHive/OpenCTI	http://soc-app-02:443	MISP instance URL
MISP_KEY	TheHive/OpenCTI	<api_key>	MISP authorization key
GRAFANA_URL	AlertManager	http://soc-app-01:3000	Grafana base URL
SMTP_HOST	Alert notifications	mail.internal.djezzy.dz	Internal mail relay
SLACK_WEBHOOK	Alert notifications	https://hooks.slack.com/...	Slack channel webhook

Table A.1: Environment Variables Reference

Appendix B: Command Quick Reference

Operation	Command	Notes
View ES cluster health	curl -s localhost:9200/_cluster/health?pretty -u elastic:pass	Green/Yellow/Red status
List ES indices	curl -s "localhost:9200/_cat/indices?v&s=index" -u elastic:pass	Size, doc count, health
ES index settings	curl -s "localhost:9200/<index>/_settings?pretty" -u elastic:pass	Replicas, refresh interval
Kafka topic list	kafka-topics.sh --bootstrap-server localhost:9092 --list	All topics in cluster
Kafka consumer groups	kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list	Active consumer groups
Kafka consumer lag	kafka-consumer-groups.sh --bootstrap-server localhost:9092 --lag	Lag, group, topic, describe
PG replication status	sudo -u postgres psql -c "SELECT * FROM pg_stat_replication;"	Current_lsn vs replay_lsn
PG active connections	sudo -u postgres psql -c "SELECT count(*) FROM pg_stat_activity;"	Current connection count
PG table sizes	sudo -u postgres psql -c "SELECT relname, pg_total_relation_size(relname) AS size FROM pg_class LIMIT 10;"	Biggest tables
Redis info	redis-cli -a <pass> INFO	Memory, keyspace, clients
Docker container stats	docker stats --no-stream	CPU, MEM, NET I/O per container
Docker logs tail	docker logs --tail 100 -f <container_name>	Follow container logs
System resource overview	htop or glances	CPU, MEM, SWAP, LOAD
Disk usage by mount	df -hT	Filesystem type, size, used%, mounted on
Top disk consumers	du -sh /data/* 2>/dev/null sort -rh head -20	Largest directories under /data
Network connections	ss -tnp grep ESTAB wc -l	Count established TCP connections
Listening ports	ss -tlnp	Services listening on each port
Systemd service status	systemctl status <service>	Running, enabled, recent logs
Journal logs	journalctl -u <service> -f --since "1 hour ago"	Service-specific logs

Table B.1: Command Quick Reference